Lambda

Introduzione alla programmazione funzionale in Java 8

Finora abbiamo visto come Java sia nato come linguaggio **orientato agli oggetti**, in cui l'attenzione principale è rivolta agli **oggetti** come contenitori di dati mutabili, mentre metodi e classi sono strumenti secondari per manipolare questi dati.
Nei test, abbiamo esplorato come **JUnit** e librerie come **Mockito** permettano di isolare le classi e simulare le dipendenze, garantendo test unitari efficaci e indipendenti, mentre test di integrazione e di componente verificano il funzionamento tra più moduli o con servizi reali.

Con l'evoluzione di Java, a partire dalla versione 8, è stato introdotto il supporto alla **programmazione funzionale**, un paradigma che consente di trattare **funzioni come oggetti**, passandole come parametri ai metodi e semplificando la gestione dei comportamenti ripetuti. Le principali novità includono:

- **Interfacce funzionali**: **interfacce con un solo metodo astratto, che definiscono un contratto per comportamenti riutilizzabili. Possono essere standard (** `Predicate<T>` **,** `Consumer<T>` **,** `Function<T,R>` **) o personalizzate.**
- **Lambda expressions**: **funzioni anonime compatte che implementano un'interfaccia funzionale, permettendo di scrivere codice più leggibile e modulare.**
- **Behavior parameterization**: **possibilità di passare il comportamento (cioè la logica da eseguire) come parametro di un metodo, separando chiaramente configurazione e azione.**
- **Metodi** `default` **nelle Collection**: **metodi come** `forEach` **,** `removeIf` **,** `replaceAll` **e** `sort` **sfruttano interfacce funzionali, permettendo di manipolare le collezioni in modo più espressivo e conciso.**

In sintesi, mentre in passato per filtrare, ordinare o trasformare dati era necessario scrivere classi o metodi specifici, con Java 8 possiamo ora **astrarre il comportamento**, passare funzioni come parametri e ridurre notevolmente la complessità del codice.
Nei paragrafi successivi approfondiremo la sintassi delle lambda, le interfacce

funzionali standard e come queste si integrano nel framework delle Collection, fornendo esempi concreti di applicazione.

Contesto storico

Come già sappiamo Java nasce nel **1995**come linguaggio fortemente **Object Oriented**, in un periodo in cui il paradigma funzionale era già presente ma non dominante nel mondo enterprise.

La progettazione di Java si basa su alcuni principi fondamentali:

- **centralità dell'oggetto;**
- **incapsulamento dello stato;**
- **mutabilità dei dati;**
- **interazione tra oggetti tramite metodi.**

Paradigma Object Oriented

Come abbiamo già sviscerato nella lezione 4 al paragrafo Approccio orientato agli oggetti (OOP), nella programmazione orientata agli oggetti:

- **l'oggetto è l'elemento centrale;**
- **l'oggetto contiene stato (dati) e comportamento (metodi);**
- **lo stato è generalmente mutabile.**

Gerarchia concettuale

- **Elemento primario** → **Oggetto (dato condiviso e modificabile)**
- **Elementi secondari** → **Classi e metodi, strumenti per creare e manipolare oggetti**

In questo modello:

- il sistema è visto come un insieme di oggetti che collaborano;
- i metodi servono a modificare o interrogare lo stato interno degli oggetti;
- la logica ruota attorno alla gestione dello stato.

Paradigma funzionale (contrapposizione concettuale)

Mentre nel paradigma funzionale la prospettiva è diversa:

- **l'elemento centrale è la funzione, non l'oggetto;**
- **i dati sono preferibilmente immutabili;**

- **l'attenzione è sul *cosa calcolare*, non su *chi possiede lo stato*.**

Qui la gerarchia si ribalta:

- **Elemento primario** → Funzione
- **Elemento secondario** → Strutture dati

Tuttavia, esistono situazioni in cui il modello puramente orientato agli oggetti risulta verboso o poco flessibile, e diventa naturale adottare una logica più vicina al paradigma funzionale.

> **Perché questo è importante per Java?**
>
> A partire da **Java 8**, il linguaggio introduce:
>
> - lambda expression
> - stream API
> - interfacce funzionali
>
> **Questi strumenti permettono di integrare concetti della programmazione funzionale in un linguaggio storicamente orientato agli oggetti.**
>
> Non si tratta di sostituire l'OOP, **ma di integrare un nuovo modo di esprimere la logica, spesso più dichiarativo e conciso.**

Esempio: `TreeSet` e `Comparator`

Un caso emblematico è l'utilizzo di `TreeSet`.

Il `TreeSet` è una collezione ordinata di elementi.
A differenza di `HashSet`, **dove l'ordinamento è basato sull'hash e non è controllabile dal programmatore**, `TreeSet` **permette di definire un criterio di ordinamento personalizzato.**

Dispone di due costruttori principali:

Java

```java
TreeSet<E>()
TreeSet<E>(Comparator<? super E> comparator)
```

Nel secondo caso, è necessario fornire un oggetto che implementi l'interfaccia `Comparator`.

Poiché `Comparator` è un'interfaccia, per utilizzarla è necessario creare una classe concreta, ad esempio:

Java

```java
public class MarcaComparator implements Comparator<Prodotto> {
    @Override
    public int compare(Prodotto p1, Prodotto p2) {
        return p1.getMarca().compareTo(p2.getMarca());
    }
}
```

E poi istanziarla:

Java

```java
TreeSet<Prodotto> t = new TreeSet<>(new MarcaComparator());
```

Problema

Questo approccio:

- **richiede la creazione di una classe dedicata;**
- **introduce codice boilerplate;**
- **è eccessivamente verboso quando il comportamento è semplice.**

Qui emerge l'esigenza di poter passare **una funzione come parametro**, senza dover definire una classe completa.

Esempio 2 : filtrare una collezione

Supponiamo di avere una classe `Mela` con attributi come:

- `colore`
- `peso`

e di voler scrivere un metodo che **filtri una collezione di mele** secondo un certo criterio.

Ad esempio:

1. filtrare le mele verdi;
2. filtrare le mele con peso maggiore di 150 grammi.

Soluzione tradizionale (approccio OOP classico)

Seguendo un approccio imperativo tradizionale, potremmo scrivere due metodi distinti:

1. **Filtrare per colore:**

Java

```java
public static List<Mela> filtraMelePerColore(List<Mela> cassetta)
{
    List<Mela> listaFiltrata = new ArrayList<Mela>();
    for (Mela m : cassetta) {
        if (m.getColore().equals("verde")) {
            listaFiltrata.add(m);
        }
    }
    return listaFiltrata;
}
```

2. **Filtrare per peso:**

Java

```java
public static List<Mela> filtraMelePerPeso(List<Mela> cassetta) {
    List<Mela> listaFiltrata = new ArrayList<Mela>();
    for (Mela m : cassetta) {
        if (m.getPeso() > 150) {
            listaFiltrata.add(m);
        }
    }
    return listaFiltrata;
}
```

Problema di questo approccio

A prima vista il codice funziona correttamente, ma presenta un problema strutturale:

- La **logica di iterazione** è identica in entrambi i metodi.
- Cambia soltanto la **condizione di filtraggio**.

Se domani volessimo filtrare:

- mele rosse,
- mele leggere,
- mele verdi e pesanti,
- mele con peso compreso tra due valori,

dovremmo continuare a scrivere nuovi metodi, duplicando ogni volta la stessa struttura di ciclo.

Questo porta a:

- duplicazione di codice;
- scarsa flessibilità;
- violazione del principio DRY (*Don't Repeat Yourself*).

Osservazione chiave

In realtà, ciò che varia non è l'algoritmo di filtraggio, ma **il criterio di selezione**.

Quindi il vero problema è:

> Come possiamo rendere parametrico il criterio di filtraggio?

Ed è proprio qui che entra in gioco la programmazione funzionale in Java:

- passare un **comportamento** come parametro;
- trattare una **funzione come dato**;
- utilizzare lambda expression per rendere il codice più generico e riutilizzabile.

Espressioni Lambda

Le **lambda expression** risolvono proprio questo problema.

Una lambda è:

- **una funzione anonima;**
- **compatta;**
- **passabile come parametro.**

Concettualmente, è simile all'operatore ternario rispetto all'if-else:

- **l'operatore ternario è una forma compatta di if-else;**
- **la lambda è una forma compatta di classe anonima che implementa un'interfaccia funzionale.**

In altri linguaggi come Python o JavaScript esistono costrutti analoghi (arrow function).

Predicato

Osservando l'esempio precedente sul filtraggio delle mele, si nota che:

- **la struttura del metodo è sempre la stessa;**
- **l'unica differenza è la condizione che determina se una mela deve essere aggiunta alla lista.**

Questa condizione restituisce un valore booleano.

In matematica, una funzione che restituisce **true o false** si chiama **predicato**.

Dal Java 7 al Java 8

In Java 7, per ogni criterio di filtraggio era necessario scrivere un metodo separato:

- uno per il colore,
- uno per il peso,
- uno per eventuali altri criteri.

Con Java 8, grazie alle **lambda expression**, è possibile:

- definire un unico metodo generico;
- passare il predicato come parametro.

Metodo generico con `Predicate`

Java 8 introduce l'interfaccia funzionale `Predicate<T>` (nel package `java.util.function`), che rappresenta proprio un predicato.

La firma del metodo diventa:

```java
public static List<Mela> filtraMele(List<Mela> cassetta, Predicate<Mela> p) {
    List<Mela> listaFiltrata = new ArrayList<>();

    for (Mela m : cassetta) {
        if (p.test(m)) {
            listaFiltrata.add(m);
        }
    }

    return listaFiltrata;
}
```

Qui:

- `Predicate<Mela>` **rappresenta una funzione che prende una** `Mela` **e restituisce un boolean;**
- `p.test(m)` **applica il criterio alla mela corrente.**

Invocazione con Lambda

Ora possiamo chiamare il metodo passando direttamente una funzione anonima:

Filtro per peso

```java
filtraMele(cassetta, (Mela m) -> m.getPeso() > 150);
```

Filtro per colore

```java
filtraMele(cassetta, (Mela m) -> m.getColore().equals("verde"));
```

La lambda:
rappresenta una funzione che:

- prende una mela come parametro;

- restituisce il risultato dell'espressione booleana.

Metodi come parametri di metodi

Il concetto chiave introdotto dalla programmazione funzionale è il seguente:

> **Una funzione può essere passata come parametro di un metodo.**

Questo significa che un metodo può accettare come argomenti:

- **tipi primitivi,**
- **oggetti,**
- **funzioni.**

Si parla quindi di programmazione secondo uno **stile funzionale**.

📋 Nuova sintassi introdotta in Java 8

Java 8 introduce una sintassi compatta per esprimere funzioni anonime: le **lambda expression**.

Questo permette di:

- evitare la creazione di classi anonime verbose;
- rendere il codice più dichiarativo;
- concentrarsi sul *cosa fare*, non sul *come implementarlo*.

✓ Vantaggi dello stile funzionale

L'uso di lambda e interfacce funzionali comporta diversi benefici:

- **Migliore modularizzazione del codice**
  Il comportamento è separato dall'algoritmo.
- **Codice più sintetico e leggibile**
  Si elimina il boilerplate tipico delle classi anonime.
- **Maggiore predisposizione al cambiamento dei requisiti**
  Nuovi criteri possono essere introdotti senza modificare il metodo di filtraggio

Elementi chiave della programmazione funzionale in Java 8

L'introduzione di **lambda expression** e **interfacce funzionali** ruota attorno a tre concetti fondamentali.

1. Functional Interface

Una *functional interface* è:

- **un'interfaccia che contiene un solo metodo astratto.**

Può eventualmente avere:

- **metodi** `default`**,**
- **metodi** `static`**,**
- **metodi privati (da Java 9 in poi),**

> **ma deve avere un solo metodo astratto, che definisce il comportamento funzionale.**

Esempio generico:

Java

```java
public interface Predicate<T> {
    boolean test(T t);
}
```

Un esempio reale fornito dalla libreria standard è `Predicate<T>` (package `java.util.function`), che rappresenta una funzione:

- **in ingresso: un oggetto di tipo** `T`
- **in uscita: un** `boolean`

Per convenzione, una functional interface può essere annotata con:

Java

```java
@FunctionalInterface
```

Questo non è obbligatorio, ma garantisce a compile-time che l'interfaccia abbia un solo metodo astratto.

2. Behavior Parameterization

La *behavior parameterization* consiste nel:

- **passare un comportamento come parametro di un metodo.**

Invece di passare solo dati (primitivi o oggetti), si passa una funzione che definisce cosa fare.

Esempio:

Java

```java
public static void esegui(String obj, Predicate<String> p) {
    System.out.println(p.test(obj));
}
```

Qui il metodo:

- **riceve una stringa** `obj`**,**
- **riceve un comportamento** `p`**,**
- **applica il comportamento tramite** `p.test(obj)`**.**

Il metodo non conosce la logica del controllo: delega la decisione al predicato.

Questo realizza la separazione tra:

- algoritmo (la struttura del metodo),
- comportamento (la condizione specifica).

3. Lambda Expression

La *lambda expression* è la nuova sintassi introdotta in Java 8 per fornire un'implementazione compatta di una functional interface.

Esempio di invocazione:

Java

```java
esegui("ciao", (String s) -> s.length() > 3);

```

La lambda:

```java
(String s) -> s.length() > 3
```

rappresenta una funzione che:

- **prende in input una `String`,**
- **restituisce un `boolean`.**

Il compilatore:

- **deduce che deve essere un `Predicate<String>`,**
- **genera automaticamente un'implementazione dell'interfaccia.**

Forma ancora più sintetica (type inference):

```java
esegui("ciao", s -> s.length() > 3);
```

Espressioni Lambda

Le **espressioni lambda** introdotte in Java 8 possono essere descritte attraverso quattro caratteristiche principali.

1. Funzioni

Sono considerate funzioni perché:

- **rappresentano un blocco di codice eseguibile;**
- **non sono legate a una specifica classe come accade per i metodi tradizionali;**
- **possono essere trattate come valori.**

Tuttavia, in Java non esiste un vero e proprio *tipo funzione*.
Le lambda esistono sempre nel contesto di una **interfaccia funzionale**.

2. Anonime

Sono prive di nome.

A differenza dei metodi tradizionali:

```java
public boolean controllo(String s) {
```

```
    return s.length() > 3;
}
```

una lambda non dichiara un identificatore per la funzione, ma solo:

- **parametri**
- **corpo**
- **valore restituito**

3. Passabili

Possono essere passate come argomenti a un metodo.

Esempio:

Java

```
esegui("ciao", s -> s.length() > 3);
```

Qui la lambda viene trattata come un oggetto che implementa una functional interface.

Questo realizza la **behavior parameterization**, ovvero il passaggio del comportamento come parametro.

4. Concise

La sintassi è molto più compatta rispetto alle classi anonime.

Versione con classe anonima:

Java

```
esegui("ciao", new Predicate<String>() {
    @Override
    public boolean test(String s) {
        return s.length() > 3;
    }
});
```

Versione con lambda:

Java

```
esegui("ciao", s -> s.length() > 3);
```

Si elimina completamente il boilerplate.

> **Nota:**
>
> **Le lambda:**
>
> - **NON introducono un nuovo paradigma nel linguaggio;**
> - **NON introducono un nuovo tipo primitivo "funzione".**
>
> Java rimane un linguaggio object-oriented.
>
> Per evitare di introdurre un *function type*, Java utilizza le **interfacce funzionali** come tipo di riferimento delle lambda.
>
> In altre parole:
>
> > **Una lambda è sempre un'istanza di una functional interface.**

Sintassi di un'espressione Lambda

La forma generale è:

Java

```
(lista_parametri) -> { corpo }
```

Dove:

- `lista_parametri` → **parametri del metodo astratto**
- `->` → **operatore lambda**
- `corpo` → **implementazione del metodo astratto**
  Inoltre il corpo può essere di 2 tipi:

1. **Espressione**
2. **Blocco di istruzioni**

La differenza dipende dal tipo di ritorno del metodo astratto dell'interfaccia funzionale.

Casi sintattici possibili

1. Un solo parametro (senza tipo esplicito)

```java
s -> s.length() > 3
```

Il tipo è dedotto dal contesto (type inference).

2. Più parametri

```java
(a, b) -> a + b
```

3. Corpo con una sola espressione

```java
s -> s.length()
```

In questo caso:

- le parentesi graffe sono opzionali
- il `return` è implicito

4. Corpo con più istruzioni

```java
s -> {
    System.out.println(s);
    return s.length() > 3;
}
```

In questo caso:

- le parentesi graffe sono obbligatorie
- il `return` è esplicito

Cosa accade a livello di compilazione

Quando si scrive una lambda:

Java

```java
s -> s.length() > 3
```

il compilatore:

1. **individua l'interfaccia funzionale attesa dal contesto (es. `Predicate<String>`);**
2. **genera una classe sintetica che implementa tale interfaccia;**
3. **crea un'istanza di quella classe.**

Concettualmente è come se: venisse creata una classe anonima, ma in modo più efficiente e trasparente.

Punto chiave da ricordare

L'espressione lambda:

- **rappresenta l'implementazione del metodo astratto di una functional interface;**
- **non esiste autonomamente;**
- **deve sempre essere associata a un tipo funzionale.**

Regole per una Functional Interface

Una interfaccia abilita l'invocazione in stile lambda **solo se rappresenta un tipo funzionale valido**.
Affinché ciò avvenga, devono essere rispettate le seguenti condizioni strutturali:

1.
    1. **Deve essere un'interfaccia (non una classe).**
       Le lambda in Java non sono funzioni autonome, ma istanze di un tipo.
       Questo tipo deve essere un'interfaccia con un unico metodo astratto.
2. **Può essere generica.**
   L'uso dei generics consente di definire interfacce funzionali riutilizzabili e tipizzate (`Predicate<T>`, `Function<T,R>`, ecc.).

3. **Deve avere esattamente un solo metodo astratto.**
   Questo è il vincolo essenziale: **la lambda rappresenta l'implementazione di quel metodo.**
4. Il metodo astratto può:
   - **avere qualsiasi tipo di ritorno,**
   - **accettare qualsiasi numero e tipo di parametri,**
   - **dichiarare eccezioni (`throws`).**

Il punto centrale è che la lambda deve poter essere associata **in modo non ambiguo** a un solo metodo astratto.

‹› Metodi che non influiscono sul conteggio

Non vengono considerati nel conteggio dei metodi astratti:

- metodi `default`
- metodi `static`
- metodi concreti ereditati da `Object`
  - `toString()`
  - `equals()`
  - `hashCode()`

Questo perché non introducono nuovi obblighi di implementazione.

Esempi: Interfaccia → Lambda corrispondente

Una lambda essere **compatibile con la firma del metodo astratto**:

- **stesso numero di parametri**
- **tipi compatibili**
- **tipo di ritorno compatibile**

La compatibilità viene verificata a compile-time tramite il meccanismo di **target typing**:

- **il compilatore deduce il tipo della lambda dal contesto in cui viene usata.**

Interfaccia con parametro e ritorno boolean

```java
public interface A {
    boolean test(Mela m);
}
```

Lambda:

```java
(Mela m) -> m.getColor().equals("red")
```

Interfaccia senza parametri

```java
public interface B {
    boolean test();
}
```

Lambda:

```java
() -> new Random().nextInt(100) > 50
```

Interfaccia con ritorno void

```java
public interface C {
    void test(Mela m);
}
```

Lambda con blocco:

```java
(Mela m) -> { System.out.println(m); }
```

Lambda ancora più sintetica (singola istruzione):

```java
```

```
(Mela m) -> System.out.println(m)
```

Interfaccia con ritorno int

```java
public interface D {
    int test(String s);
}
```

Lambda:

```java
(String s) -> s.length()
```

Interfaccia con più parametri

```java
public interface E {
    int test(int a, int b);
}
```

Lambda:

```java
(int v1, int v2) -> v1 * v2
```

Interfacce funzionali nelle collection

Come abbiamo anticipato poco sopra, Java 8 introduce l'annotazione:

```java
@FunctionalInterface
```

Serve a **marcare esplicitamente un'interfaccia come funzionale**.

- **È facoltativa.**
- **Se presente, il compilatore verifica che l'interfaccia rispetti le regole:**
  - **esattamente un solo metodo astratto.**

Se la regola non è rispettata → errore di compilazione.

🔗 **Analogia con `@Override`**

Dal punto di vista pratico svolge lo stesso ruolo di `@Override`:

- **non è obbligatoria,**
- **ma è buona pratica usarla,**
- **permette al compilatore di intercettare errori progettuali.**

Quindi la regola fondamentale di un interfaccia funzionale è:

- **un interfaccia si dice funzionale SE E SOLO SE possiede esattamente un metodo astratto**

Può inoltre contenere:

- metodi `default`
- metodi `static`
- metodi ereditati da `Object`

L'importante è che il numero totale effettivo di metodi astratti rimanga uno.

⚠ **Osservazione progettuale importante**

La restrizione del "singolo metodo astratto" non è arbitraria.

Serve a garantire che:

- **una lambda rappresenti un comportamento ben definito;**
- **il compilatore possa determinare senza ambiguità quale metodo venga implementato;**
- **il codice rimanga leggibile e semanticamente chiaro.**

Se un'interfaccia avesse due metodi astratti, una lambda non potrebbe sapere quale dei due implementare.

Gerarchie delle interfacce funzionali

La regola del **"un solo metodo astratto"** deve essere rispettata considerando **anche i metodi ereditati**.

Non conta solo ciò che è dichiarato localmente nell'interfaccia, ma l'insieme totale dei metodi astratti risultanti dalla gerarchia.

Caso 1 — Interfaccia figlia che aggiunge un nuovo metodo

Java

```java
@FunctionalInterface
interface InterfaceA {
    void metodo1();
}

@FunctionalInterface
interface InterfaceB extends InterfaceA {
    void metodo2();
}
```

Analisi

- `InterfaceA`
  → 1 metodo astratto → è funzionale.
- `InterfaceB`
  → eredita `metodo1()`
  → dichiara `metodo2()`
  → totale metodi astratti effettivi = 2

Quindi `InterfaceB` **non è funzionale**.
L'annotazione `@FunctionalInterface` produce errore.

Perché?

Perché una lambda può implementare un solo metodo.
Se l'interfaccia ne richiede due, **il compilatore non può associare la lambda in modo non ambiguo.**

Diverso il caso invece in cui l'interfaccia figlia ridefinisce (cioè fa l'overriding) lo **stesso metodo**, allora, solo in quel caso, il metodo astratto resta uno solo anche per l'interfaccia figlia che può dirsi *interfaccia funzionale*.

In tal caso entrambe possono essere funzionali.

Caso 2 — Ridefinizione dello stesso metodo (override)

Caso diverso:

```java
@FunctionalInterface
interface InterfaceA {
    void metodo1();
}

@FunctionalInterface
interface InterfaceB extends InterfaceA {
    @Override
    void metodo1();
}
```

Qui:

- `InterfaceB` **non introduce un nuovo metodo astratto.**
- **Sta semplicemente ridefinendo la stessa firma.**

Il numero totale di metodi astratti effettivi rimane 1.

In questo caso:

- `InterfaceA` **è funzionale**
- `InterfaceB` **è funzionale**

ⓘ Cosa significa “metodi astratti effettivi”?

Il compilatore considera:

- metodi dichiarati nell’interfaccia;
- metodi astratti ereditati;
- eventuali override che unificano firme compatibili.

Non conta i metodi:

- `default`
- `static`
- concreti ereditati da `Object`

Il criterio non è “quanti metodi vedo scritti”, ma:

> Quanti obblighi astratti deve soddisfare una classe che implementa questa interfaccia?

Se l'obbligo è uno solo → l'interfaccia è funzionale.

Se sono più di uno → non lo è.

Il principio generale è:

> **Un'interfaccia è funzionale se e solo se, dopo aver considerato tutta la gerarchia di ereditarietà, possiede esattamente un solo metodo astratto effettivo.**

"Effettivo" significa:

- non duplicato,
- non `default`,
- non `static`,
- non ereditato da `Object`.

Lambda e interfacce funzionali: esempi generali e applicazioni nelle Collection

Le **lambda expression** in Java 8 permettono di implementare interfacce funzionali in modo sintetico, senza creare classi anonime verbose.
Ogni lambda è associata al **metodo astratto di una interfaccia funzionale**.

Lambda assegnate a interfacce generiche

Prima di Java 8, esistevano già diverse interfacce funzionali standard, come:

- `Runnable` → metodo `void run()`
- `Comparator<T>` → metodo `int compare(T o1, T o2)`
- `Callable<V>` → metodo `V call()`

Con le lambda possiamo assegnarle direttamente a oggetti di queste interfacce:

Java

```java
Runnable r = () -> System.out.println("funziona!");
Thread t = new Thread(r);
t.start();
```

- `() -> System.out.println("funziona!")` è una funzione anonima
- Viene passata come implementazione del metodo `run()`
- Si elimina il boilerplate delle classi anonime
  Allo stesso modo, per un `Comparator`:

Java

```java
Comparator<String> comp = (s1, s2) -> s1.length() - s2.length();
List<String> lista = Arrays.asList("uno","due","tre");
Collections.sort(lista, comp);
```

Qui la lambda implementa il metodo `compare(T o1, T o2)`.

Interfacce funzionali nel framework Collections

A partire da Java 8, il framework delle **Collection** è stato esteso con **metodi** `default` **già implementati** che accettano interfacce funzionali.

Questo rappresenta un punto fondamentale: **Java ha introdotto lo stile funzionale senza rompere la compatibilità con il codice esistente**, permettendo di passare comportamenti come parametri anziché dover creare classi anonime o implementazioni dedicate.

Metodi principali introdotti

`Iterable.forEach(Consumer<? super T> action)`

- **Permette di eseguire un'operazione su ogni elemento della collezione.**
- Accetta un **Consumer:**
  - **un'interfaccia funzionale che prende un parametro e non restituisce nulla.**
- Tipicamente usato per operazioni con **effetti collaterali**, come stampare valori, modificare variabili esterne, loggare informazioni.

Java

```java
lista.forEach(e -> System.out.println(e));
```

Qui la lambda `(e -> System.out.println(e))` implementa il metodo astratto di `Consumer<T>`.

`Collection.removeIf(Predicate<? super T> filter)`

- **Permette di rimuovere elementi che soddisfano un certo criterio.**
- Accetta un **Predicate**, **cioè un'interfaccia funzionale che prende un parametro e ritorna un booleano.**
- Il booleano stabilisce se l'elemento deve essere rimosso (`true`) o mantenuto (`false`).
  Esempio:

Java

```java
lista.removeIf(e -> e.isEmpty());
```

Qui la lambda `(e -> e.isEmpty())` restituisce `true` per rimuovere gli elementi vuoti.

`List.replaceAll(UnaryOperator<T> operator)`

- **Permette di trasformare tutti gli elementi di una lista secondo un criterio.**
- Accetta un `UnaryOperator<T>`:
  - **un'interfaccia funzionale che prende un elemento e restituisce un elemento dello stesso tipo.**
- **È utile per applicare trasformazioni in-place senza creare nuove collezioni**.
  Esempio:

Java

```java
lista.replaceAll(s -> s.toUpperCase());
```

Qui tutti gli elementi della lista vengono convertiti in maiuscolo.

`List.sort(Comparator<? super T> c)`

- Permette di **ordinare gli elementi** secondo un criterio definito.
- Accetta un **Comparator:**
  - cioè un'interfaccia funzionale con metodo:

```java
int compare(T o1, T o2);
```

- La lambda restituisce un valore negativo, zero o positivo per indicare l'ordine.
  Esempio:

```java
lista.sort((a, b) -> a.length() - b.length());
```

Qui gli elementi vengono ordinati in base alla lunghezza.

> **Nota:**
>
> Anche se `Comparator` contiene metodi default come `reversed()` o `thenComparing()`, ha **un solo metodo astratto effettivo** (`compare`) e perciò è una interfaccia funzionale compatibile con le lambda.

Esempio pratico di uso delle interfacce funzionali nelle Collection

Consideriamo una lista di stringhe che rappresentano dei fiori:

```java
ArrayList<String> fiori = new ArrayList<>();
fiori.add("rosa");
fiori.add("narciso");
fiori.add("margherita");
fiori.add("iris");
```

Possiamo operare sulla lista usando **metodi che accettano interfacce funzionali**:

1. `removeIf(Predicate)` → rimuove gli elementi che soddisfano una certa condizione.

```java
fiori.removeIf(s -> s.endsWith("a"));
```

- Lista risultante: `narciso, iris`

- `Predicate<T>`: `T -> boolean`

2. `replaceAll(UnaryOperator)` → trasforma tutti gli elementi della lista secondo una funzione.

```java
fiori.replaceAll(s -> s.toUpperCase());
```

- Lista risultante: `NARCISO, IRIS`
- `UnaryOperator<T>`: `T -> T`

3. `sort(Comparator)` → ordina la lista secondo un criterio.

```java
fiori.sort((s, t) -> s.compareTo(t));
```

- Lista risultante: `IRIS, NARCISO`

4. `forEach(Consumer)` → esegue un'operazione su ciascun elemento

```java
fiori.forEach(s -> System.out.println(s));
```

Stampa:

```nginx
IRIS
NARCISO
```

- `Consumer<T>`: `T -> void`

Sintesi delle principali interfacce funzionali in uso nelle Collection:

- `Predicate<T>` → `T -> boolean`
- `UnaryOperator<T>` → `T -> T`
- `BiFunction<T,U,R>` → `(T,U) -> R`
- `Consumer<T>` → `T -> void`

Sintassi delle espressioni lambda: type-checked e type-inference

Le **lambda expression** in Java sono **type-checked:**

- **cioè il compilatore verifica la correttezza dei tipi dei parametri e del valore di ritorno rispetto all'interfaccia funzionale che la lambda implementa.**

Esempio: type-checked

Supponiamo di avere l'interfaccia funzionale `MelaPredicate`:

Java

```java
public interface MelaPredicate {
    boolean test(Mela m);
}
```

Il metodo `test` prende un oggetto `Mela` e restituisce un booleano.
Possiamo assegnare una lambda compatibile:

Java

```java
MelaPredicate p = (Mela m) -> m.getPeso() > 150;
```

- **Il parametro `m` deve essere di tipo `Mela`**
- **Il risultato della lambda deve essere `boolean`**
- **Se assegnassimo un tipo diverso o restituissero un valore incompatibile, il compilatore genera un errore**

Questo garantisce che **le lambda siano sicure a compile-time**, riducendo errori rispetto all'uso di classi anonime.

Type inference (inferenza del tipo)

Java 8 permette di **semplificare ulteriormente la sintassi** usando l'inferenza dei tipi del compilatore:

- **Se il compilatore può determinare il tipo del parametro dalla dichiarazione della variabile o dal contesto, il tipo può essere omesso nella lambda.**

- **Se ci sono due o più parametri, occorrono sempre le parentesi tonde per racchiuderli.**

Esempio semplificato:

```java
MelaPredicate p = m -> m.getPeso() > 150;
```

- **Qui il compilatore inferisce automaticamente che `m` è di tipo `Mela`**
- **La lambda è più compatta e leggibile, senza perdere la sicurezza dei tipi**

Per lambda con più parametri:

```java
BiPredicate<Mela, Integer> p2 = (m, soglia) -> m.getPeso() > soglia;
```

- Parentesi tonde obbligatorie per racchiudere più parametri
- **La tipologia di `m` e `soglia` viene dedotta dal contesto (`BiPredicate<Mela, Integer>`)**

In sintesi:

1. Le lambda sono **type-checked** →
   - **il compilatore verifica che i tipi dei parametri e il tipo di ritorno siano coerenti con l'interfaccia funzionale.**
2. Possono sfruttare **type inference** →
   - **i tipi dei parametri possono essere omessi quando il compilatore li può dedurre, rendendo la lambda più concisa e leggibile.**
3. Lambda con più parametri richiedono **parentesi tonde**, mentre con un solo parametro le parentesi possono essere opzionali.

Questa combinazione di sicurezza e sintesi rende le lambda uno strumento potente per **scrivere codice più pulito e modulare** rispetto alle classi anonime tradizionali.

Interfacce funzionali di Java 8

Andiamo ad analizzare le principali interfacce funzionali di Java partendo dalle interfacce che specializzano i tipi primitivi

Specializzazioni per tipi primitivi

Interfaccia	Chiamata lambda	Descrizione	Specializzazione
Predicate	`T -> boolean`	**Valuta una condizione su un valore e restituisce `true` o `false`.** Utilizzato per filtri o condizioni logiche.	IntPredicate, LongPredicate, DoublePredicate
Consumer	`T -> void`	**Esegue un'operazione su un valore senza restituire nulla (`void`).** Tipico per operazioni con effetti collaterali, es. stampa o aggiornamenti di stato.	IntConsumer, LongConsumer, DoubleConsumer
Function<T, R>	`T -> R`	**Trasforma un valore in un altro tipo `R`.** Utile per map/filter in stream o conversioni.	IntFunction, LongFunction, DoubleFunction
Function specializzata	`T -> R`	**Conversione da un tipo primitivo ad un altro tipo primitivo o generico, evitando boxing/unboxing.**	IntToDoubleFunction, IntToLongFunction, LongToDoubleFunction, LongToIntFunction, DoubleToIntFunction, DoubleToLongFunction

Supplier, Unary, Binary e BiConsumer/BiFunction

Interfaccia	Chiamata lambda	Descrizione	Specializzazione
Supplier	`() -> T`	**Fornisce un valore di tipo T senza parametri.** Utile per generare valori “lazy” o predefiniti.	BooleanSupplier, IntSupplier, LongSupplier, DoubleSupplier
UnaryOperator	`T -> T`	**Opera su un singolo valore e restituisce un risultato dello stesso tipo.** Esempio: incremento, scaling, calcolo matematico.	IntUnaryOperator, LongUnaryOperator, DoubleUnaryOperator
BinaryOperator	`(T, T) -> T`	**Opera su due valori dello stesso tipo e restituisce un risultato dello stesso tipo.** Utile per somme, moltiplicazioni o combinazioni.	IntBinaryOperator, LongBinaryOperator, DoubleBinaryOperator
BiConsumer<L, R>	`(L, R) -> void`	**Accetta due valori (oggetto + primitivo) e esegue un’operazione senza ritorno.** Tipico per aggiornare campi di un oggetto basati su un primitivo.	ObjIntConsumer, ObjLongConsumer, ObjDoubleConsumer
BiFunction<T, U, R>	`(T, U) -> R`	**Riceve due argomenti e restituisce un valore di tipo R**. Utile per calcoli	ToIntBiFunction<T, U>, ToLongBiFunction<T, U>, ToDoubleBiFunction<T, U>

Interfaccia	Chiamata lambda	Descrizione	Specializzazione
		combinati o trasformazioni di coppie di dati.	
BiPredicate<T, U>	`(T, U) -> boolean`	**Valuta una condizione logica su due argomenti, restituendo** `true/false`**.** Ideale per filtri su coppie di oggetti o confronto tra valori.	BiPredicate<T, U>

Sintesi e osservazioni:

1. Le interfacce **specializzate per i tipi primitivi** (int, long, double) evitano **boxing/unboxing**, migliorando le performance rispetto alle versioni generiche (`Predicate<Integer>`, `Function<Double, R>`, ecc.).
2. I nomi seguono uno schema chiaro:
   - `Unary` → **un parametro, stesso tipo di ritorno**
   - `Binary` → **due parametri, stesso tipo di ritorno**
   - `BiConsumer` / `BiFunction` → **due parametri, operazioni con o senza ritorno**
   - `Supplier` → **nessun parametro, fornisce un valore**
3. L'uso in **stream API** **o operazioni funzionali** semplifica l'implementazione di filtri, mappe, riduzioni o side-effect, mantenendo il codice conciso e leggibile.

Interfacce funzionali di Java 8 e Collections

Con Java 8, il linguaggio ha introdotto un set completo di **interfacce funzionali**, molte delle quali sfruttano i **generics** per aumentare la flessibilità e la riusabilità del codice. Questo permette di scrivere metodi generici che accettano qualsiasi tipo di oggetto, rendendo il codice più modulare e riutilizzabile.

Tuttavia, i generics:

- **non funzionano direttamente con i tipi primitivi (** `int`**,** `long`**,** `double`**). Per utilizzare i primitivi nei contesti generici, Java richiede l'uso dei**

wrapper (`Integer`, `Long`, `Double`) oppure sfrutta l'autoboxing:

- **che converte automaticamente il primitivo nel wrapper corrispondente.**

> ⚠ **Attenzione:**
>
> l'uso dei wrapper, sia esplicito che tramite autoboxing, comporta un piccolo **costo computazionale**, dovuto alla creazione degli oggetti wrapper.
> In scenari ad alte prestazioni può essere rilevante.

Per questo motivo, Java 8 fornisce anche **specializzazioni delle interfacce funzionali** per i tipi primitivi, che **permettono di operare direttamente su `int`, `long` e `double` senza passare dai wrapper**:

Tipo primitivo	Interfaccia funzionale specializzata	Tipo generico equivalente
`int`	`IntPredicate`, `IntConsumer`, `IntFunction`	`Predicate<Integer>`, `Consumer<Integer>`, `Function<Integer, R>`
`long`	`LongPredicate`, `LongConsumer`, `LongFunction`	`Predicate<Long>`, `Consumer<Long>`, `Function<Long, R>`
`double`	`DoublePredicate`, `DoubleConsumer`, `DoubleFunction`	`Predicate<Double>`, `Consumer<Double>`, `Function<Double, R>`

Creare una propria interfaccia funzionale

Java permette di definire interfacce personalizzate da usare con le lambda. Alcuni schemi comuni:

1. **Predicato** → **restituisce boolean**:

Java

```java
public interface MyFunctional<T> {
    boolean myMethod(T type);
}
```

2. **Consumer → restituisce void**:

```java
public interface MyFunctional<T> {
    void myMethod(T type);
}
```

3. **Supplier → restituisce un valore**:

```java
public interface MyFunctional<T> {
    T myMethod();
}
```

4. **UnaryOperator → prende e restituisce lo stesso tipo**:

```java
public interface MyFunctional<T> {
    T myMethod(T type);
}
```

5. **Function → prende un tipo e restituisce un altro**:

```java
public interface MyFunctional<T,R> {
    R myMethod(T type);
}
```

6. **BiFunction → prende due parametri e restituisce un risultato**:

```java
public interface MyFunctional<T,U,R> {
    R myMethod(T type, U other);
}
```

Questi schemi sono alla base della programmazione funzionale in Java e permettono di creare comportamenti **riutilizzabili e passabili come argomenti**, facilitando la modularità e la leggibilità del codice.